

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



PROYECTO INVENTARIO Y TIENDA DE COMPRAS

Javier Chávez 23132

Dally Ramirez - 241035

Denis Rodríguez - 21151

Diego Sandoval – 231977

Adrián López - 231361

Ingeniería de software 1 – CC 3090

Guatemala, 2026

Resumen:

El presente proyecto corresponde a la segunda fase del desarrollo de un sistema de software para Gold Fashion S.A., una empresa dedicada a la comercialización de joyería y perfumes que cuenta con tres locales físicos: Luxor, Glamour y Victoria. El trabajo se enfoca en la definición de las características generales del sistema, así como en la ideación, prototipado y modelación de una solución tecnológica que permita centralizar la gestión de inventarios y ventas entre sus sucursales.

La necesidad del proyecto surge de la falta de integración entre los procesos actuales de inventario y venta, los cuales se realizan de forma manual o mediante herramientas no unificadas, dificultando el control del stock, la coordinación entre locales y la toma de decisiones estratégicas. En un contexto donde el comercio digital y la eficiencia operativa son factores clave para la competitividad, resulta indispensable contar con un sistema centralizado y escalable.

El objetivo principal es diseñar un sistema que permita administrar inventarios en tiempo real y ofrecer una plataforma web de ventas que mejore la experiencia tanto del personal como de los clientes. Como objetivos específicos se busca definir las funcionalidades del sistema, identificar actores e historias de usuario, y modelar la solución utilizando herramientas de ingeniería de software y metodologías centradas en el usuario.

Introducción:

Descripción de la entidad:

Gold Fashion S.A. es una empresa dedicada a la venta de joyería y perfumes, con presencia física a través de tres locales comerciales: Luxor, Glamour y Victoria. Cada uno de estos locales ofrece productos como anillos, collares, pulseras, relojes y perfumes, muchos de ellos de marcas reconocidas o de edición limitada. La empresa atiende tanto a clientes recurrentes como a nuevos compradores, combinando ventas presenciales con ventas realizadas a través de redes sociales.

La estructura operativa de la empresa incluye administradores responsables del control general del negocio, vendedores encargados de la atención al cliente y del registro de ventas, y procesos de inventario que actualmente se manejan de forma independiente en cada local. El proyecto se enmarca en el área de gestión de inventarios y ventas, con énfasis en la centralización de la información y la optimización de procesos entre las distintas sucursales.

Descripción de la idea:

La idea general del proyecto es diseñar un sistema centralizado de inventario dinámico, acompañado de una plataforma web de ventas, que permita a Gold Fashion S.A. gestionar de manera eficiente los productos de sus tres locales desde una única solución tecnológica. El sistema busca reflejar en tiempo real la disponibilidad de productos, registrar ventas, generar reportes y facilitar la compra para los clientes.

La problemática principal identificada es la falta de integración entre los locales, lo que provoca inconsistencias en el inventario, errores en la disponibilidad de productos y dificultades para obtener información confiable sobre ventas y stock. Además, la ausencia de una plataforma web formal limita la experiencia del cliente y reduce las oportunidades de crecimiento del negocio. Esta situación evidencia la necesidad de una solución que unifique los procesos y mejore tanto la gestión interna como la interacción con los clientes.

Objetivos:

Objetivo General:

Definir y modelar una solución tecnológica que permita centralizar la gestión de inventarios y ventas de Gold Fashion S.A., mejorando la eficiencia operativa y la experiencia del usuario.

Objetivos Específicos:

- Describir las características generales del sistema a desarrollar para Gold Fashion S.A.
- Aplicar técnicas de ideación y prototipado para proponer soluciones alineadas a las necesidades del negocio.
- Identificar los actores y las historias de usuario que interactuarán con el sistema.
- Modelar el sistema mediante casos de uso, mapas de historias de usuario y requisitos no funcionales.
- Establecer una base sólida para la implementación del sistema en fases posteriores del proyecto.

Design Thinking

Prototipos

Durante el desarrollo del proyecto se elaboraron diferentes versiones de prototipos con el objetivo de visualizar la interfaz del sistema propuesto para la gestión de ventas y

control de inventario de la empresa Gold Fashion S.A., que administra los locales Luxor, Glamour y Victoria.

Los prototipos fueron diseñados utilizando la herramienta Figma, permitiendo representar de manera visual las funcionalidades principales del sistema antes de su desarrollo final.

Cada iteración del prototipo incorporó mejoras basadas en la retroalimentación obtenida por los miembros del equipo y posibles usuarios del sistema, permitiendo refinar la experiencia de usuario y mejorar la organización de las funcionalidades.

Versiones

Versión 1: <https://www.figma.com/design/ZP53vNMwyg9UD7PyEefSLj/GoldFashion-S.A.-Prototype-1?node-id=0-1&t=d1WMvyePe27Hz3Jr-1>

La primera versión del prototipo se enfocó en representar la estructura general del sistema. En esta etapa se diseñaron las pantallas principales que permiten visualizar el flujo básico del sistema, incluyendo el inicio de sesión, el panel principal y las funciones iniciales de gestión de productos e inventario.

Objetivo:

El objetivo de esta primera versión fue establecer una base visual del sistema y definir la navegación entre las principales funcionalidades.

Versión 2:

<https://www.figma.com/design/8OFBhm3KbZRdKW25ndm96s/GoldFashion-S.A.-Prototype-2?node-id=10059-1603&t=dHbznpb7q3gE9W2y-1>

En la segunda versión del prototipo buscamos algo más user friendly, algo más personalizable y más simple en términos de su contenido. Con una mentalidad más minimalista en cuanto a lo mostrado al usuario en primera experiencia.

Objetivo:

Mejorar la usabilidad del sistema y ampliar la cobertura de funcionalidades necesarias para la gestión del negocio.

Retroalimentación obtenida

Durante la evaluación de los prototipos se recopilaron observaciones relacionadas con la facilidad de uso del sistema, la organización de la información y la claridad de las funcionalidades disponibles.

Principales observaciones:

- Simplificar algunos formularios de registro.
- Mejorar la visualización del inventario disponible.
- Incluir filtros para facilitar la búsqueda de productos.
- Mostrar información resumida de ventas en el panel principal.

Estas observaciones permitieron mejorar progresivamente el diseño del sistema en las siguientes versiones del prototipo.

Cambios en requisitos derivados de los prototipos

Requisito	Cambio realizado
Gestión de productos	Se agregaron filtros de búsqueda
Registro de ventas	Se simplificó el proceso de registro
Inventario	Se agregó visualización de stock disponible
Dashboard	Se añadieron indicadores de ventas

Análisis

Lista de requisitos funcionales y sus historias de usuario

ID Requisito	Requisito Funcional	Historia de Usuario Asociada
RF01	El sistema debe permitir a los usuarios iniciar sesión mediante credenciales registradas.	Como usuario del sistema, quiero iniciar sesión, para acceder a las funcionalidades del sistema de forma segura.
RF02	El sistema debe permitir registrar nuevos productos en el inventario.	Como administrador, quiero registrar nuevos productos, para mantener actualizado el inventario de la empresa.
RF03	El sistema debe permitir consultar el listado de productos disponibles.	Como empleado, quiero consultar los productos disponibles, para verificar la disponibilidad de inventario.

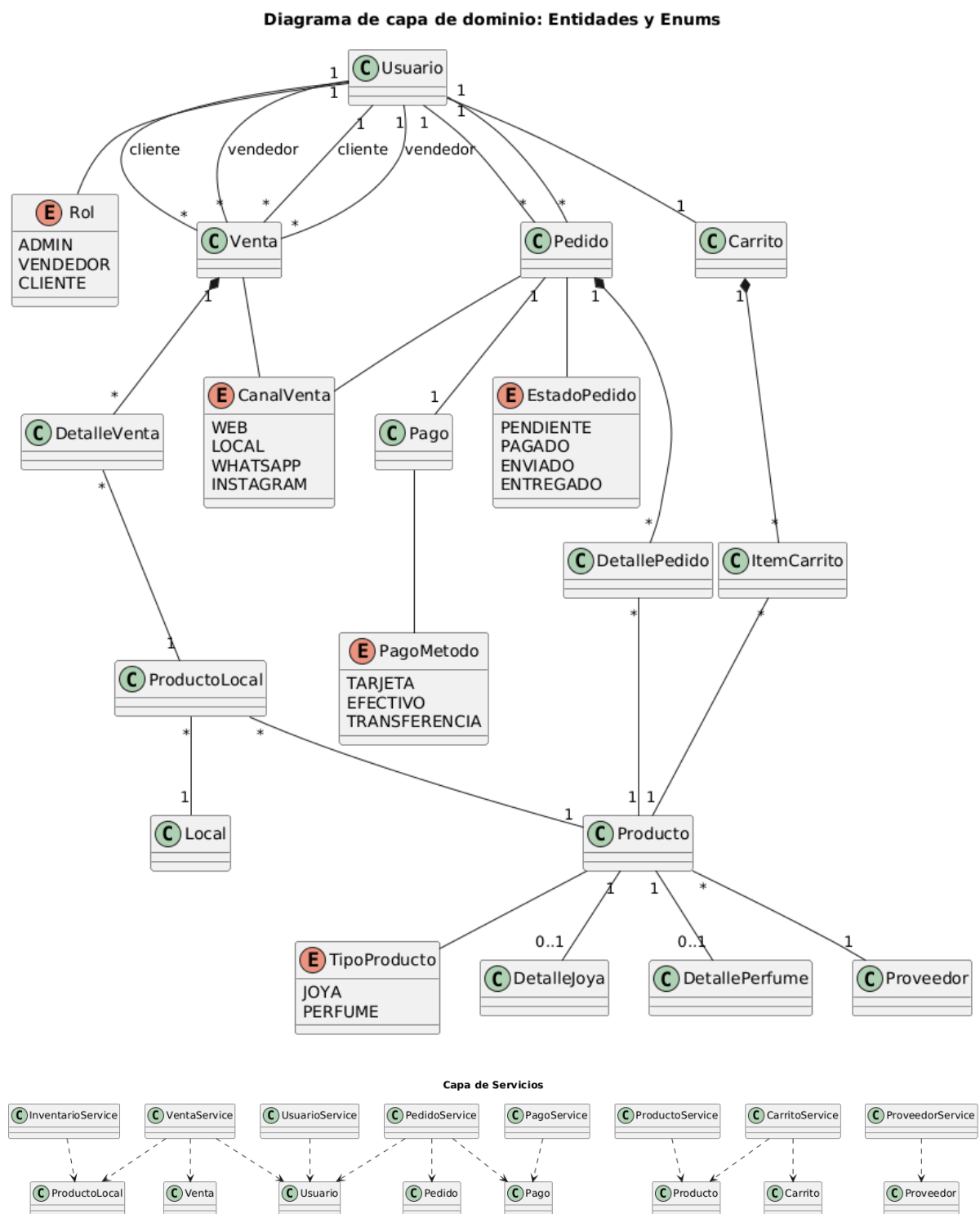
RF04	El sistema debe permitir actualizar la información de los productos.	Como administrador, quiero editar la información de los productos, para corregir o actualizar datos cuando sea necesario.
RF05	El sistema debe permitir eliminar productos del inventario.	Como administrador, quiero eliminar productos, para retirar artículos que ya no se comercializan.
RF06	El sistema debe permitir registrar ventas de productos.	Como vendedor, quiero registrar ventas, para llevar control de las transacciones realizadas.
RF07	El sistema debe permitir registrar clientes.	Como vendedor, quiero registrar clientes, para mantener información de contacto y compras.
RF08	El sistema debe permitir consultar el historial de ventas.	Como administrador, quiero consultar el historial de ventas, para analizar las transacciones realizadas.
RF09	El sistema debe permitir visualizar reportes de ventas.	Como administrador, quiero visualizar reportes de ventas, para analizar el rendimiento del negocio.
RF10	El sistema debe mostrar el nivel de inventario disponible de cada producto.	Como empleado, quiero ver el stock disponible, para saber cuándo es necesario reabastecer productos.

Clases preliminares

Usuario.

- UsuarioService.
- Producto.
- ProductoService.
- TipoProducto (**enum**).
- DetalleJoya.
- DetallePerfume.
- Venta.
- DetalleVenta.
- VentaService.
- Pedido.
- EstadoPedido (**enum**).
- DetallePedido.
- PedidoService.
- Carrito.
- CarritoService.
- ItemCarrito.
- CanalVenta (**enum**).
- Local.
- ProductoLocal.
- Pago.
- PagoService.
- InventarioService.
- PagoMetodo (**enum**).
- Rol (**enum**).
- Proveedor.
- ProveedorService.

Diagrama de clases



Descripción de las clases

Entidades: Representan a los actores y elementos más importantes del negocio.

Encapsulan los datos principales del dominio.

- **Usuario:** Representa a las personas que interactúan con el sistema. Gestiona la autenticación y diferencia los permisos de acuerdo al rol.
- **Producto:** Representa un producto del stock. Tiene la información general que todos los productos tienen en común, como precio, nombre, tipo, etc.
- **DetalleJoya:** Tiene los atributos específicos de los productos de joyería como kilataje, material, tipo de baño etc. Necesaria porque sería complicado manejar todo en Producto ya que joyería y perfumes son muy diferentes.
- **DetallePerfume:** Tiene los atributos específicos de los perfumes, como volumen y tipo (EDT, EDP, Parfum, etc.). Necesaria porque joyería y perfumería son muy diferentes y sería complicado manejar todo en una clase.
- **Venta:** Representa una transacción terminada en el sistema, puede ser en la tienda, mediante redes sociales o la página web. Ayuda a tener un historial de precios, ya que estos pueden cambiar en el tiempo.
- **DetalleVenta:** Representa el conjunto de los productos incluidos en una venta, contiene la cantidad y precio unitario. Sirve para tener un historial completo que se pueda revisar después.
- **Pedido:** Representa un pedido de productos realizado por un cliente. Permite gestionar todo el proceso antes de la venta, tiene los estados: pendiente, pagado, enviado, entregado.
- **DetallePedido:** Representa el conjunto de productos en un pedido. Permite procesar los pedidos antes de que sean una venta, asegurando que se puedan realizar confirmaciones.
- **Carrito:** Representa la selección de productos que un cliente selecciona antes de hacer el pedido. Permite hacer el proceso de compra más flexible.
- **ItemCarrito:** Representa a un producto que está dentro del carrito y la cantidad seleccionada. Permite manejar varios productos en un carrito.
- **Local:** Representa a un local físico. Es la base para el control del inventario.
- **ProductoLocal:** Representa la cantidad de un producto en un local específico, servirá para manejar el inventario.
- **Pago:** Representa la transacción de un pedido o venta. Sirve para gestionar la información como monto, método de pago y estado del pago.
- **Proveedor:** Representa a los proveedores de productos. Permite gestionar el stock y relacionar cada producto con su proveedor.

Services: Encapsulan la lógica del negocio, ejecutan los procesos y operaciones que se necesitan

- **UsuarioService:** Gestiona las operaciones de los usuarios, como login, asignación de roles y autenticación.
- **ProductoService:** Gestiona la administración de productos, creación, actualización, eliminación y cualquier consulta necesaria.\

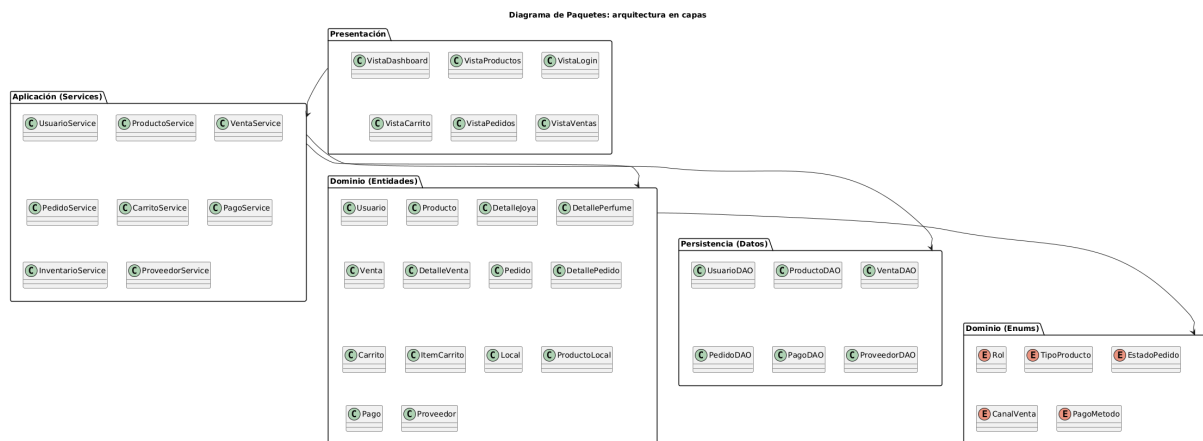
- **VentaService:** Procesa las ventas, valida el stock, registra transacciones y actualiza el stock.
- **PedidoService:** Gestiona los cambios de estado de los pedidos, y maneja desde su creación hasta que se convierten en ventas.
- **CarritoService:** Permite gestionar las operaciones del carrito, como agregar y eliminar productos del carrito, y consultarlos.
- **PagoService:** Gestiona el proceso de pagos: validación, confirmación y actualización de estado del pago.
- **InventarioService:** Controla el stock del sistema. Permite verificar disponibilidad en el stock, actualizarlo y gestionar el reabastecimiento.
- **ProveedorService:** Gestiona la información de los proveedores, como registro, actualización y cualquier consulta necesaria.

Enums: Permiten definir los valores controlados de las categorías y estados del sistema, Con ellas eliminamos errores por valores mal introducidos, se garantiza la integridad de los datos porque solo hay opciones concretas disponibles.

- **TipoProducto:** Define las categorías de productos. Permite diferenciar los tipos entre joyería, perfumes, y es fácil escalarlo para otros productos.
- **EstadoPedido:** Define los estados de un pedido en el sistema, ayuda a controlar el ciclo: pendiente -> pagado -> enviado -> entregado (El flujo puede variar dependiendo de lo que pida la empresa).
- **CanalVenta:** Indica el origen de la venta (tienda física, página web, redes sociales).
- **MetodoPago:** Define los métodos de pago que acepte la empresa.
- **Rol:** Define los roles de usuarios en el sistema para gestionar permisos y accesos.

Esta organización permite que los datos y lógica estén separados, además de mantener los valores controlados. También permite escalar mejor y que sea más fácil darle mantenimiento al código.

Diagrama de paquetes



Descripción de cada paquete y sus componentes

Paquete Presentación

Contiene las clases responsables de la interacción directa con los usuarios del sistema.

Componentes:

- VistaLogin
- VistaDashboard
- VistaProductos
- VistaCarrito
- VistaPedidos
- VistaVentas

Responsabilidades:

- mostrar información al usuario
- recibir datos ingresados por el usuario
- enviar solicitudes a la capa de servicios

Paquete Aplicación (Services)

Contiene las clases responsables de gestionar la lógica de negocio del sistema y coordinar las operaciones entre la presentación y el dominio.

Componentes:

- UsuarioService
- ProductoService
- PedidoService
- VentaService
- CarritoService
- PagoService

- InventarioService
- ProveedorService

Responsabilidades:

- procesar solicitudes provenientes de la interfaz
- aplicar reglas de negocio
- coordinar operaciones entre entidades
- gestionar el flujo de procesos como ventas, pedidos y pagos

Paquete Dominio (Entidades)

Contiene las clases que representan las entidades principales del sistema, modelando los elementos del negocio.

Componentes:

- Usuario
- Producto
- DetalleJoya
- DetallePerfume
- Venta
- DetalleVenta
- Pedido
- DetallePedido
- Carrito
- ItemCarrito
- Local
- ProductoLocal
- Pago
- Proveedor

Responsabilidades:

- representar la estructura lógica del negocio
- almacenar y organizar la información del sistema
- servir como base para las operaciones del sistema

Paquete Dominio (Enums)

Contiene los tipos enumerados que definen valores fijos utilizados en el sistema.

Componentes:

- Rol
- TipoProducto
- EstadoPedido
- CanalVenta
- PagoMetodo

Responsabilidades:

- restringir los valores posibles en el sistema
- garantizar la consistencia de los datos
- facilitar la implementación de reglas de negocio

Paquete Persistencia

Contiene las clases encargadas del acceso a la base de datos.

Componentes:

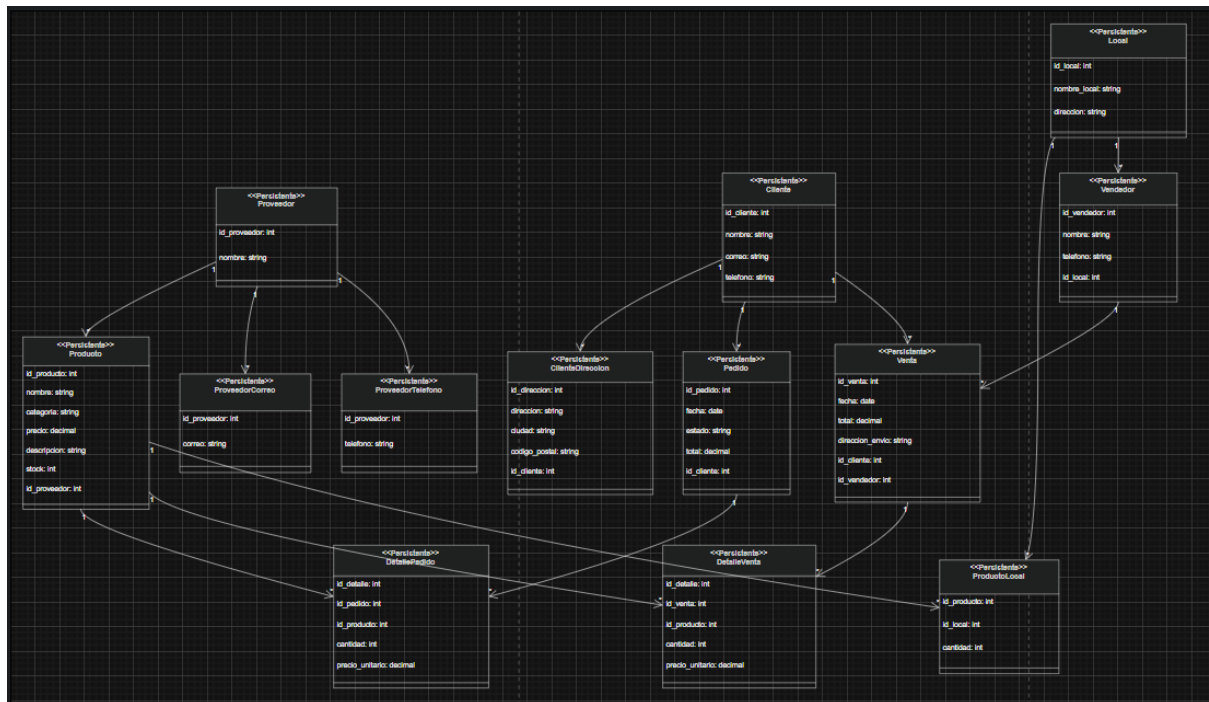
- UsuarioDAO
- ProductoDAO
- PedidoDAO
- VentaDAO
- PagoDAO
- ProveedorDAO

Responsabilidades:

- guardar información en la base de datos
- consultar información almacenada
- actualizar registros existentes
- abstraer el acceso a datos del resto del sistema

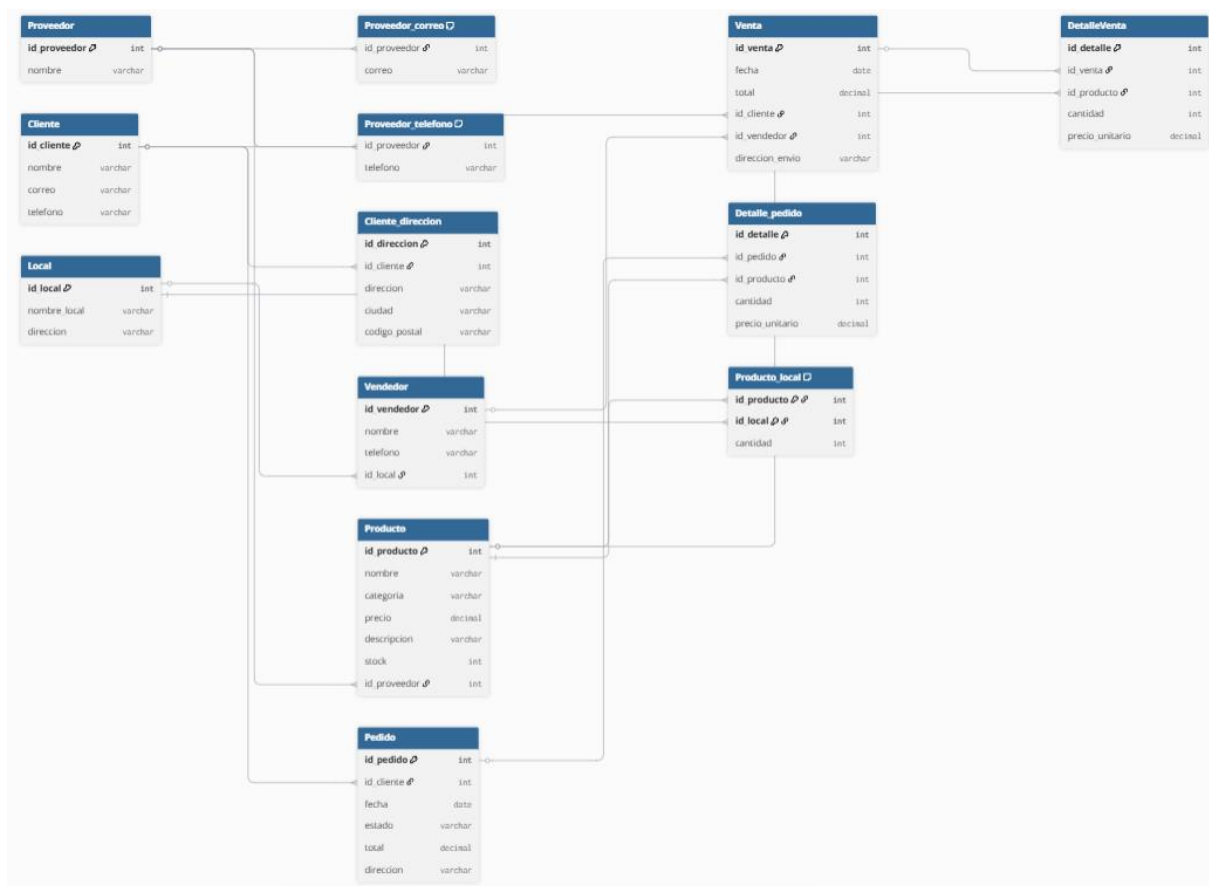
Persistencia de datos

Diagrama de clases persistentes



<https://drive.google.com/file/d/1DKQw8YtNdlz2KqRSVo8Jq-QsoKeUv5cd/view?usp=sharing>

Diagrama entidad relación



<https://dbdiagram.io/d/proyecto01db1-69b8707a78c6c4bc7af9db58>

Diseño

Estimación de dimensiones del sistema

Cantidad probable de usuarios

El sistema será utilizado por distintos tipos de usuarios:

- Administradores: 2 – 3
- Vendedores por local: 3 – 5
- Personal administrativo: 2 – 3

Considerando múltiples sucursales, se estima un total de:

20 a 30 usuarios registrados

Usuarios concurrentes

Debido al funcionamiento simultáneo de los locales:

- 5 a 10 usuarios concurrentes

Tiempo de respuesta

Se establecen los siguientes tiempos máximos aceptables:

- Consultas (productos, clientes, inventario): ≤ 2 segundos
- Registro de ventas/pedidos: ≤ 3 segundos
- Respuesta del chatbot (IA): ≤ 4 segundos

Escalabilidad

Se estima que el sistema manejará:

- 500 – 1000 productos
- 2000 – 5000 clientes
- 10 000 – 20 000 ventas anuales

Con crecimiento progresivo, se proyecta un almacenamiento de:

200 MB a 1 GB de datos en los primeros años

El sistema deberá permitir escalar tanto en usuarios como en volumen de datos sin degradar su rendimiento.

Disponibilidad

El sistema debe garantizar disponibilidad durante el horario operativo con una disponibilidad mínima del 99%.

Selección de tecnología

Se evaluaron las siguientes alternativas tecnológicas:

Java + Spring Boot

Python + Django

JavaScript (Node.js + Express + React)

Descripción detallada con ventajas y desventajas

Alternativa 1: Java + Spring Boot

Ventajas

- Alta robustez para sistemas empresariales
- Arquitectura estructurada
- Seguridad integrada

Desventajas

- Mayor complejidad de desarrollo
- Menor flexibilidad para interfaces modernas
- Mayor tiempo de implementación

Alternativa 2: Python + Django

Ventajas

- Desarrollo rápido
- Framework completo

Desventajas

- Menor rendimiento en alta concurrencia
- Menor flexibilidad en frontend moderno

Alternativa 3: Arquitectura híbrida con IA (Node.js + React + FastAPI)

Esta alternativa incorpora un servicio adicional para funcionalidades inteligentes (chatbot).

Componentes:

Frontend: React

Backend principal: Node.js + Express

Servicio de IA: FastAPI (Python)

Ventajas

- Separación clara de responsabilidades
- Permite integrar funcionalidades de inteligencia artificial
- Escalable (servicios independientes)
- FastAPI ofrece alto rendimiento para procesamiento de IA

- Flexibilidad para futuras mejoras (recomendaciones, análisis, etc.)

Desventajas

- Mayor complejidad arquitectónica
- Necesidad de comunicación entre servicios (API REST)
- Mayor esfuerzo de integración

Tecnología seleccionada

Entre todos seleccionamos una arquitectura híbrida basada en microservicios ligeros, compuesta por:

React (Frontend)

Node.js + Express (Backend principal)

FastAPI (Servicio de IA – Chatbot)

Justificación de la elección

La elección se fundamenta en los siguientes aspectos:

Permite una interfaz moderna e intuitiva mediante React.

Node.js proporciona un backend eficiente para manejar múltiples usuarios concurrentes.

FastAPI permite integrar funcionalidades de inteligencia artificial sin afectar el rendimiento del sistema principal.

La separación en servicios independientes mejora la escalabilidad y mantenibilidad.

Facilita la evolución futura del sistema, permitiendo incorporar nuevas funcionalidades inteligentes.

Esta arquitectura responde adecuadamente a los requisitos no funcionales definidos, especialmente en términos de rendimiento, escalabilidad y disponibilidad.

Tecnología para el almacenamiento persistente

Las opciones consideradas fueron:

- MySQL
- PostgreSQL

- MongoDB

MySQL

Ventajas

- Alto rendimiento en aplicaciones web
- Fácil instalación y configuración
- Amplia documentación
- Buena integración con Node.js

Desventajas

- Menor cumplimiento estricto de estándares SQL
- Limitaciones en consultas complejas y avanzadas

PostgreSQL

Ventajas

- Cumplimiento estricto del estándar SQL
- Alta integridad y consistencia de los datos
- Soporte para consultas complejas
- Excelente manejo de relaciones entre entidades
- Escalabilidad para sistemas en crecimiento
- Soporte para funciones avanzadas (triggers, procedimientos almacenados)

Desventajas

- Configuración inicial más compleja
- Requiere mayor conocimiento técnico para su administración

MongoDB

Ventajas

- Alta flexibilidad (modelo NoSQL)
- Escalabilidad horizontal

Desventajas

- No es adecuado para sistemas altamente relacionales
- Menor control de integridad referencial

- No se adapta bien a modelos como ventas, pedidos y relaciones múltiples
- Gestor de base de datos seleccionado

Nuestra elección

PostgreSQL

Justificación de la elección

La elección de PostgreSQL se fundamenta en los siguientes aspectos:

El sistema presenta un modelo altamente relacional (clientes, productos, ventas, pedidos, proveedores), lo cual requiere un gestor que garantice la integridad de los datos.

PostgreSQL ofrece un mejor manejo de relaciones complejas en comparación con otras alternativas.

Permite escalar el sistema a medida que crece el volumen de información.

Proporciona herramientas avanzadas que facilitan la implementación de reglas de negocio directamente en la base de datos.

Es adecuado para integrarse tanto con el backend en Node.js como con servicios adicionales como FastAPI.

En consecuencia, PostgreSQL se considera la opción más adecuada para garantizar la consistencia, escalabilidad y confiabilidad del sistema.

Planificación e informe de gestión

Desglose de tareas

Tarea	Descripción	Fecha inicio	Fecha fin
Revisión de requisitos	Revisar requisitos funcionales y no funcionales definidos en el corte anterior	10/03/2026	11/03/2026
Actualización de requisitos funcionales	Ajustar requisitos según retroalimentación obtenida de los prototipos	11/03/2026	12/03/2026

Desarrollo de prototipos (Iteración 2 y 3)	Elaborar nuevas versiones del prototipo del sistema basadas en las historias de usuario	12/03/2026	15/03/2026
Validación de prototipos	Presentar prototipos a posibles usuarios y recopilar observaciones	15/03/2026	16/03/2026
Identificación de clases preliminares	Definir las clases principales del sistema a partir de los requisitos funcionales	16/03/2026	18/03/2026
Elaboración del diagrama de clases	Diseñar el diagrama de clases inicial del sistema	18/03/2026	20/03/2026
Diseño de paquetes del sistema	Organizar las clases en paquetes para mantener cohesión y bajo acoplamiento	20/03/2026	21/03/2026
Diseño de persistencia de datos	Definir clases persistentes y modelo de base de datos	21/03/2026	23/03/2026
Elaboración del diagrama entidad-relación	Diseñar el modelo relacional de la base de datos	23/03/2026	24/03/2026
Selección de tecnologías	Evaluar lenguajes, frameworks y gestor de base de datos	24/03/2026	25/03/2026
Redacción del documento final del corte 3	Integrar todos los apartados y revisar el documento	25/03/2026	27/03/2026

Lista de tareas asignadas

Esto se incluye en los formularios LOGT que serán entregados por cada integrante en la carpeta de formularios del repositorio.